离屏画布

当画布绘制元素变多的情况下，画布的性能可能会变差，这时我们可以使用“离屏画布”这个优化措施去改善。

① 离屏画布（Offscreen Canvas）在内存中开辟一个新的缓冲区进行图像渲染，当你在离屏画布上进行渲染操作时，这些操作并不会立即反映到屏幕上。只有当你将离屏画布的内容复制到一个可见的画布上时，这些渲染操作才会显示出来。这种方式可以避免不必要的屏幕刷新，从而提高渲染效率。

② 离屏画布可以结合Web Worker实现在主线程之外的工作线程上进行渲染，避免主线程的阻塞，提高渲染性能。

③ 如果有一些复杂的图形或文本需要重复使用，可以先在离屏画布上进行渲染，然后复用这个渲染结果，可以避免重复的渲染操作。

思考：上面提到离屏画布是通过开辟一个新的缓冲区进行图像渲染，这和“双缓冲”是一样的吗？

解答：

两者的实现和用途是不一样的，主要是用途不一样。

实现上的不一样就不多说啦，但是假如你要在浏览器中手动实现canvas的双缓冲机制，其实实现方法和离屏画布没有什么区别。

在用途上：

- 双缓冲：可以提高渲染性能，但核心用途是解决画面闪烁和撕裂问题。
- 离屏画布：既然离屏画布可以模拟实现双缓冲，当然也是可以用于解决画面闪烁和撕裂问题，但是能做并不意味着是其核心用途，不管是结合Web Worker中进行渲染，不阻塞主线程，还是复用复杂图像，其核心用途都是为了提高渲染性能。

`WebWorker`

HTML5 提供的一种在浏览器后台独立运行的 JavaScript 线程。它允许开发者在主线程之外执行 JavaScript，从而避免长时间运行的脚本阻塞页面或影响用户交互。

`Web Worker` 有三个关键点：

① 并行处理：在主线程之外执行脚本，不会阻塞用户界面。

② 独立环境：运行在其自己的全局环境中，它不能直接访问主线程的全局环境或 DOM。

③ 消息传递：主线程和worker线程直接传递消息，使用 `postMessage` 方法发送消息，通过 `onmessage` 接收消息。

`Web Worker` 相信大家或多或少都听说过或用过，就不再过多介绍，了解更多可以查阅MDN，如果想了解一下它如果封装用起来更方便可以看看我之前在掘金写的一篇文章：

限时领掘金会员
立即前往 >

https://juejin.cn/post/7215226343712604221

开源 | Web Workers这样封装才用得爽 - 掘金

为保证用户页面体验，大量的计算和离屏渲染会通过Web Workers实现，但Web Workers原生API用起来麻麻地(不怎么样)，因此我给它做了一次“大改造”，并且…

`WebWorker` 既然不能访问DOM那又如何与canvas结合使用呢？

① `OffscreenCanvas API`

我们一般创建游离画布都是通过 `let canvas = document.createElement('canvas')` 的方式创建，如果你打算使用这样的游离画布作为离屏画布，你是体会到它“不阻塞主线程”的优势，因为 `canvas` 是文档内的节点（不管是否游离），所以是无法在 `Web Worker` 中使用 `Canvas`，使用不恰当甚至会出现渲染能力没提高，反而可能导致“卡帧”的情况。

相比于 `canvas`，`OffscreenCanvas API` 可以创建一个脱离屏幕渲染的canvas对象，和文档没有直接关系，它是在内存中创建的，不会直接影响或被DOM影响，这也使得它不管是窗口环境还是worker环境中均能有效使用。

使用方式也非常简单：

```
const offscreen = new OffscreenCanvas(/* width /256, / height */256);
// 其他绘制操作和canvas没有任何区别
const ctx = offscreen.getContext('2d');
ctx.fillStyle = 'red';
ctx.fillRect(0, 0, 50, 50);
```

区别于 `canvas`，`OffscreenCanvas` 没有 `toBlob()` API，只有 `convertToBlob()` API，但是作用是一样的，相当于改了个名。

https://codepen.io/huanjinliu/embed/gOBaorJ?editors=0011

② 通过 `getImageData()` 获取像素数据后传给worker线程处理，避免过大运算导致主线程阻塞，处理后再将处理结果重新传回给主线程使用。具体了解可前往：像素操作